D3_Deep_Learning_Xarxes_Neuronals_en_detall

February 17, 2026

1 D3. Deep Learning: Xarxes Neuronals en detall

1.1 Machine Learning vs Deep Learning

El Machine Learning va canviar completament el paradigma en que la Inteligència Artificial enfocava la resolució de problemes. En lloc de programar les regles, es proporcionen al sistema exemples d'entrades i les seves sortides corresponents i deixem que l'algoritme descobreixi les regles per si mateix.

El Machine Learning tradicional requeix que els desenvolupadors decideixin quines característiques són importants (procés anomenat *feature engineering*).

El Deep Learning va un pas més enllà i utilitza xarxes neuronals amb múltiples capes (d'aquí ve el nom de "profund") que poden aprendre automàticament quines característiques són importants sense la nostra intervenció.

1.2 Xarxes Neuronals vs Deep Learning

Una xarxa neuronal és una arquitectura inspirada en el funcionament del cervell humà: neurones artificials organitzades en capes (entrada, ocultes, sortida) connectades per pesos que s'ajusten durant l'entrenament.

El Deep Learning, en canvi és una tècnica d'aprenentatge que utilitza xarxes neuronals profundes (múltiples capes ocultes) per aprendre automàticament.

Exemple:

```
del = keras.Sequential([
    layers.Dense(64, activation='relu'),     # definim 64 neurones de la primera capa
    layers.Dense(32, activation='relu'),     # definim 32 neurones de la segona capa
    layers.Dense(10, activation='softmax')   # 10 possibles sortides
])
```

Aquesta codi defineix una xarxa neuronal simple, que seria la típica que utilitzaríem per classificar dígits.

El Deep Learning és el procés que passa després:

```
model.fit(X_train, y_train, epochs=50)
```

Quan la xarxa neuronal s'entrena amb `fit()`, passa la màgia del deep learning: aprèn automàticament quines característiques són importants per ella mateixa.

Cada capa s'especialitza en detectar patrons diferents: les primeres capes capturen patrons simples (línies horitzontals, verticals, cantonades), mentre les últimes capes combinen aquests blocs bàsics per reconèixer patrons complexos.

1.3 Tècniques avançades: AutoML i Neural Architecture Search (NAS)

En tècniques avançades com AutoML automatitzem tant el disseny d'arquitectures neuronals (NAS) automatitatzant tot el procés: només s'han de proporcional dades i el *target* i el sistema descobreix l'arquitectura, els *features* i els pesos òptims fent ús de tècniques avançades com algoritmes genètics, reinforcement learning o gradient descent.

Veiem un exemple que compara AutoML amb Deep Learning on configurem la xarxa neuronal:

[1]:
```
!pip install autokeras
```

```
Collecting autokeras
  Downloading autokeras-3.0.0-py3-none-any.whl.metadata (5.6 kB)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-
packages (from autokeras) (26.0)
Collecting keras-tuner>=1.4.0 (from autokeras)
  Downloading keras_tuner-1.4.8-py3-none-any.whl.metadata (5.6 kB)
Requirement already satisfied: keras>=3.0.0 in /usr/local/lib/python3.12/dist-
packages (from autokeras) (3.10.0)
Requirement already satisfied: dm-tree in /usr/local/lib/python3.12/dist-
packages (from autokeras) (0.1.9)
Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-
packages (from keras>=3.0.0->autokeras) (1.4.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(from keras>=3.0.0->autokeras) (2.0.2)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages
(from keras>=3.0.0->autokeras) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages
(from keras>=3.0.0->autokeras) (0.1.0)
Requirement already satisfied: h5py in /usr/local/lib/python3.12/dist-packages
(from keras>=3.0.0->autokeras) (3.15.1)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages
(from keras>=3.0.0->autokeras) (0.18.0)
Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.12/dist-
packages (from keras>=3.0.0->autokeras) (0.5.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-
packages (from keras-tuner>=1.4.0->autokeras) (2.32.4)
Collecting kt-legacy (from keras-tuner>=1.4.0->autokeras)
  Downloading kt_legacy-1.0.5-py3-none-any.whl.metadata (221 bytes)
Requirement already satisfied: grpcio in /usr/local/lib/python3.12/dist-packages
(from keras-tuner>=1.4.0->autokeras) (1.76.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-
packages (from keras-tuner>=1.4.0->autokeras) (5.29.6)
Requirement already satisfied: attrs>=18.2.0 in /usr/local/lib/python3.12/dist-
packages (from dm-tree->autokeras) (25.4.0)
```

```
Requirement already satisfied: wrapt>=1.11.2 in /usr/local/lib/python3.12/dist-
packages (from dm-tree->autokeras) (2.1.1)
Requirement already satisfied: typing-extensions~=4.12 in
/usr/local/lib/python3.12/dist-packages (from grpcio->keras-
tuner>=1.4.0->autokeras) (4.15.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests->keras-
tuner>=1.4.0->autokeras) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-
packages (from requests->keras-tuner>=1.4.0->autokeras) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests->keras-
tuner>=1.4.0->autokeras) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests->keras-
tuner>=1.4.0->autokeras) (2026.1.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from rich->keras>=3.0.0->autokeras)
(4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from rich->keras>=3.0.0->autokeras)
(2.19.2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-
packages (from markdown-it-py>=2.2.0->rich->keras>=3.0.0->autokeras) (0.1.2)
Downloading autokeras-3.0.0-py3-none-any.whl (124 kB)
                         124.7/124.7 kB
3.8 MB/s eta 0:00:00
Downloading keras_tuner-1.4.8-py3-none-any.whl (129 kB)
                         129.4/129.4 kB
4.1 MB/s eta 0:00:00
Downloading kt_legacy-1.0.5-py3-none-any.whl (9.6 kB)
Installing collected packages: kt-legacy, keras-tuner, autokeras
Successfully installed autokeras-3.0.0 keras-tuner-1.4.8 kt-legacy-1.0.5
```

[2]:
```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import autokeras as ak
```

[3]:
```
# DADES MNIST
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()
X_train = X_train.reshape(-1, 28, 28, 1).astype('float32') / 255
X_test = X_test.reshape(-1, 28, 28, 1).astype('float32') / 255
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/mnist.npz
11490434/11490434            0s
0us/step
```

[4]:
```
model = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Flatten(),
    layers.Dense(64, activation='relu', input_shape=(784,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

[5]:
```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(X_train, y_train, epochs=5, batch_size=128,
          validation_data=(X_test, y_test), verbose=1)

print(f"Accuracy: {model.evaluate(X_test, y_test, verbose=0)[1]:.4f}")
model.summary()
```

```
Epoch 1/5
469/469             10s 16ms/step -
accuracy: 0.7680 - loss: 0.7855 - val_accuracy: 0.9413 - val_loss: 0.1994
Epoch 2/5
469/469             5s 9ms/step -
accuracy: 0.9451 - loss: 0.1929 - val_accuracy: 0.9561 - val_loss: 0.1485
Epoch 3/5
469/469             4s 9ms/step -
accuracy: 0.9607 - loss: 0.1369 - val_accuracy: 0.9621 - val_loss: 0.1232
Epoch 4/5
469/469             3s 6ms/step -
accuracy: 0.9687 - loss: 0.1046 - val_accuracy: 0.9671 - val_loss: 0.1058
Epoch 5/5
469/469             2s 4ms/step -
accuracy: 0.9720 - loss: 0.0922 - val_accuracy: 0.9697 - val_loss: 0.0997
Accuracy: 0.9697
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0

dense (Dense)	(None, 64)	50,240
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 10)	330

Total params: 157,952 (617.00 KB)

Trainable params: 52,650 (205.66 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 105,302 (411.34 KB)

[6]:
```
model = keras.Sequential([
    # Capa 1: Aprèn vores, gradients
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D(),

    # Capa 2: Aprèn formes (cercles, línies, angles)
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(),

    # Capa 3: Aprèn parts (peces de dígits)
    layers.Conv2D(128, (3,3), activation='relu'),

    # Classificació final
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

```
/usr/local/lib/python3.12/dist-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

[7]:
```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(X_train, y_train, epochs=5, batch_size=128,
```

```
          validation_data=(X_test, y_test), verbose=1)

print(f"Accuracy: {model.evaluate(X_test, y_test, verbose=0)[1]:.4f}")
model.summary()
```

```
Epoch 1/5
469/469             56s 114ms/step -
accuracy: 0.8683 - loss: 0.4550 - val_accuracy: 0.9815 - val_loss: 0.0589
Epoch 2/5
469/469             57s 122ms/step -
accuracy: 0.9843 - loss: 0.0536 - val_accuracy: 0.9891 - val_loss: 0.0337
Epoch 3/5
469/469             78s 114ms/step -
accuracy: 0.9897 - loss: 0.0322 - val_accuracy: 0.9891 - val_loss: 0.0335
Epoch 4/5
469/469             83s 115ms/step -
accuracy: 0.9922 - loss: 0.0242 - val_accuracy: 0.9921 - val_loss: 0.0251
Epoch 5/5
469/469             80s 111ms/step -
accuracy: 0.9936 - loss: 0.0201 - val_accuracy: 0.9924 - val_loss: 0.0231
Accuracy: 0.9924
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 128)	73,856
flatten_1 (Flatten)	(None, 1152)	0
dense_3 (Dense)	(None, 128)	147,584
dense_4 (Dense)	(None, 10)	1,290

Total params: 724,640 (2.76 MB)

Trainable params: 241,546 (943.54 KB)

```
Non-trainable params: 0 (0.00 B)


Optimizer params: 483,094 (1.84 MB)
```

1.4 Funcionament d'una Xarxa Neuronal

Una xarxa neuronal és, bàsicament, un sistema que aprèn a fer prediccions mitjançant un procés iteratiu d'assaig i error. Aquest procés es pot dividir en cinc passos fonamentals que es repeteixen una i altra vegada fins que el sistema arriba a fer prediccions prou bones.

- **Pas 1 :** proporcionar dades a la xarxa neuronal. Aquestes dades poden ser de qualsevol tipus: caracterìstuqes d'una casa, píxels d'una imatge, paraules d'un text, etc. sempre que puguin ser representades en format numèric.
- **Pas 2:** fer una predicció. La xarxa fa una predicció utilitzant els seus paràmtres interns (pesos i biases). Al principi, aquest paràmetres tenen una valor aleatori, i provablement la predicció serà incorrecte.
- **Pas 3:** comparar amb resultat real. La diferpnecia entre els dos valors s'anomena error.
- **Pas 4:** ajust de paràmetres. Utilitza l'error calcular per intentar fer una millor predicció la propera vegada. Aquest procés utilitza algoritmes matemètics per el *gradient descent* i *backpropagation*.
- **Pas 5:** repetir el procés. Aquest 4 passos es repeteixen moltes vegades. A cada iteració, la xarxa millora una mica més.

El cicle continua fins que: * L'error és prou petit. * Hem fet totes les iteracions planificades. * La xarxa deixa de millorar.

[8]:
```python
# AutoML màgic
clf = ak.ImageClassifier(max_trials=1, overwrite=True)
clf.fit(X_train, y_train, epochs=3, validation_data=(X_test, y_test))

# Resultats (típic)
print(f"Accuracy: {clf.evaluate(X_test, y_test)[1]:.4f}")
best_model = clf.export_model()
best_model.summary()
```

```
Trial 1 Complete [00h 09m 34s]
val_loss: 0.03841746225953102

Best val_loss So Far: 0.03841746225953102
Total elapsed time: 00h 09m 34s
Epoch 1/3
1875/1875              167s 88ms/step
- accuracy: 0.9020 - loss: 0.3111 - val_accuracy: 0.9841 - val_loss: 0.0492
Epoch 2/3
```

```
1875/1875              159s 84ms/step
- accuracy: 0.9776 - loss: 0.0722 - val_accuracy: 0.9862 - val_loss: 0.0406
Epoch 3/3
1875/1875              160s 86ms/step
- accuracy: 0.9816 - loss: 0.0600 - val_accuracy: 0.9863 - val_loss: 0.0399

/usr/local/lib/python3.12/dist-packages/keras/src/saving/saving_lib.py:802:
UserWarning: Skipping variable loading for optimizer 'adam', because it has 2
variables whereas the saved optimizer has 14 variables.
  saveable.load_own_variables(weights_store.get(inner_path))

313/313             8s 23ms/step -
accuracy: 0.9842 - loss: 0.0463
Accuracy: 0.9868

Model: "functional"
```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 28, 28, 1)	0
cast_to_float32 (CastToFloat32)	(None, 28, 28, 1)	0
normalization (Normalization)	(None, 28, 28, 1)	3
conv2d (Conv2D)	(None, 26, 26, 32)	320
conv2d_1 (Conv2D)	(None, 24, 24, 64)	18,496
max_pooling2d (MaxPooling2D)	(None, 12, 12, 64)	0
dropout (Dropout)	(None, 12, 12, 64)	0
flatten (Flatten)	(None, 9216)	0
dropout_1 (Dropout)	(None, 9216)	0
dense (Dense)	(None, 10)	92,170
classification_head_1 (Softmax)	(None, 10)	0

Total params: 110,989 (433.55 KB)

Trainable params: 110,986 (433.54 KB)

Non-trainable params: 3 (16.00 B)

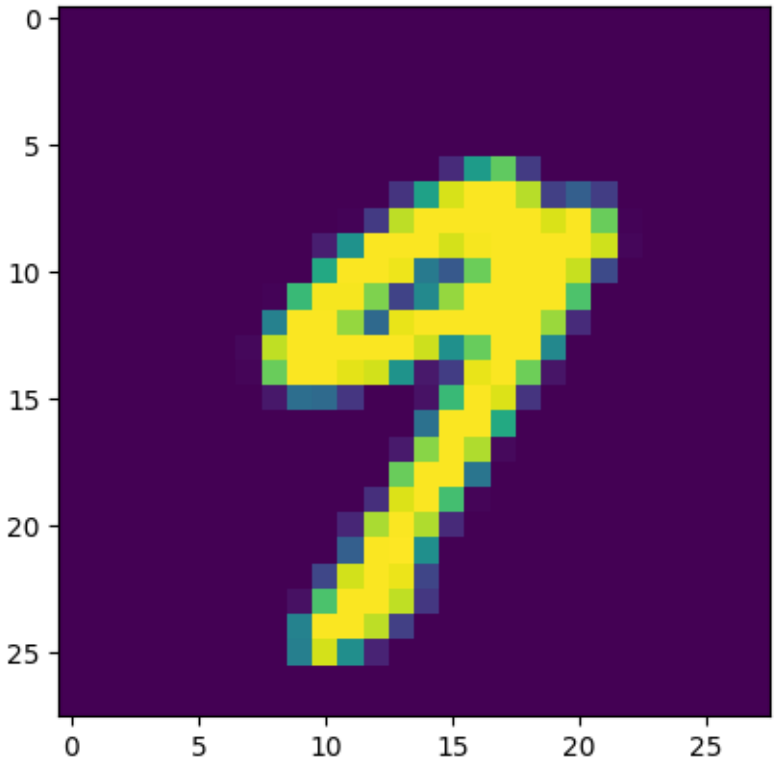
[9]:
```
best_model.save('classificador.keras')
```

[10]:
```
from tensorflow.keras.models import load_model
import matplotlib.pyplot as plt
import numpy as np

loaded_model = load_model(
    'classificador.keras',
    custom_objects=ak.CUSTOM_OBJECTS)
prediction = loaded_model.predict(X_test[20:21])
etiqueta_predita = np.argmax(prediction)  # Ex: 7
print(f"Etiqueta predita: {etiqueta_predita}")

plt.imshow(X_test[20].reshape(28, 28))
```

```
1/1             0s 115ms/step
Etiqueta predita: 9
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/saving/saving_lib.py:802:
UserWarning: Skipping variable loading for optimizer 'adam', because it has 14
variables whereas the saved optimizer has 2 variables.
  saveable.load_own_variables(weights_store.get(inner_path))
```

[10]:
```
<matplotlib.image.AxesImage at 0x7ecf956b0a40>
```



RECURSOS: Exemple interactiu per entendre millor el funcionament d'una xarxa neuronal: https://playground.tensorflow.org/